

© Joshua M. Willman 2020

J. M. Willman, *Beginning PyQt*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-5857-6_7

7. Creating GUIs with Qt Designer

Joshua M. Willman¹

(1) Hampton, VA, USA

The previous chapters have focused on learning how to manually code GUIs using PyQt. This was done intentionally so that you could have a better fundamental understanding of the code and processes used to create simple applications. Chapters [2](#) and [3](#) showed you how to create your own GUI from scratch. In Chapter [4](#), you learned about layouts and how to arrange widgets by coding them yourself. You saw how to create applications with menus and toolbars in Chapter [5](#) and how to style their look in Chapter [6](#).

While setting up and arranging GUIs yourself gives you more control over the design process, not everyone will need or want to take the time to do so. Fortunately, Qt provides a great application for setting up the layouts and designing main windows, widgets, or dialogs. **Qt Designer** is a graphical interface filled with Qt widgets and other tools used for building GUIs. Using the Qt Designer application's drag and drop interface, you are able to create and customize your own dialogs, windows, and widgets.

The widgets and other applications you create using Qt Designer integrate with programmed code, using Qt's signals and slots mechanism, so that you can easily assign behavior to widgets and graphical elements. This means that rather than focusing most of your time on layout and design, you can get into coding the functionality of an application much faster.

In Chapter [7](#), you will

- Find out about the Qt Designer user interface
- Create an application in Qt Designer, including how to set layouts, edit object properties, connect signals and slots, and generate Python code
- Learn about the `QFrame` class for containing other widgets
- Be introduced to a couple new Qt classes including `QPalette` and

`QIntValidator`

TipFor references or more help beyond the scope of this chapter, check out the Qt Documentation for Qt Designer at <https://doc.qt.io/qt-5/qtdesigner-manual.html>.

Getting Started with Qt Designer

Once you have installed PyQt, the first thing you need to do is to launch the Qt Designer application. After opening Qt Designer, you will see a

graphical user interface for creating your own GUIs like the one in Figure

[7-1](#).

NoteFor more information about downloading and launching Qt Designer for Windows, MacOS, and Linux, refer to Appendix A.

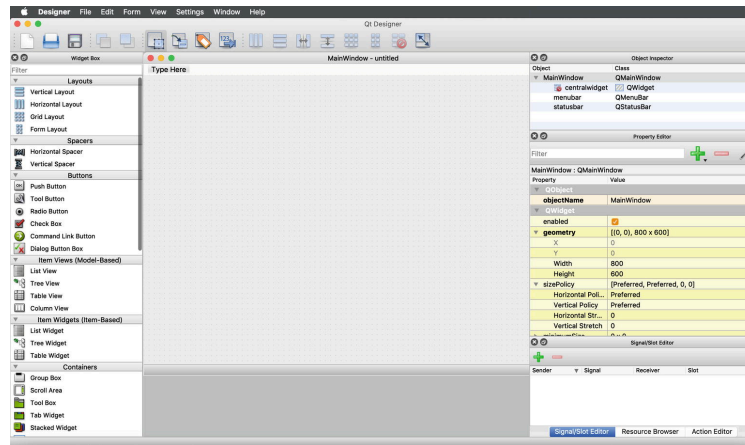


Figure 7-1 The Qt Designer user interface

TipIf you are using a stand-alone version of Qt Designer, then you can change the configuration of Qt Designer under `Settings`. Locate the `Preferences` menu, and in the dialog box that appears, you can change the appearance of Qt Designer from a Multi Top-Level Windows user interface to one using Docked Windows. Otherwise, the multilevel layout is only available if you use Qt Creator, the integrated development environment (IDE) for working with Qt.

Before you create your first application, let's get to know the different menus, tools, and modes that are displayed in the main window in Figure

[7-1](#).

Exploring Qt Designer's User Interface

When you first open up Qt Designer, you will notice a dialog in the center of the window with the title `New Form`. This dialog can be seen in Figure [7-2](#). From here you can select a template for creating a main window, a widget, or different kinds of dialog boxes. You can also choose what kinds of widgets to add to your project's layout. Once you have selected a template and the application's size, an empty window, also known as a **form**, will appear for you to modify.

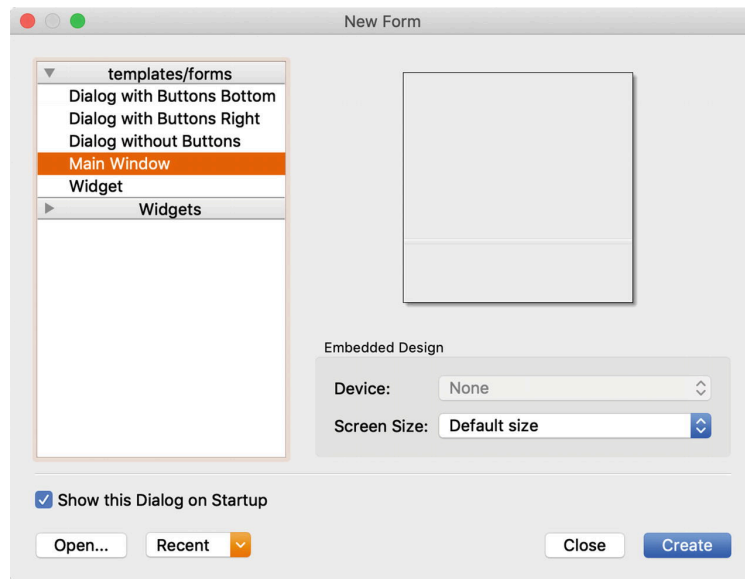


Figure 7-2 The New Form dialog box for selecting what type of application to build

At the top of the main window in Figure 7-1, you will notice Qt Designer's menubar and toolbar for managing and editing your GUI. On the left side of the main window is the `Widget Box` dock widget, shown in Figure 7-3, which provides an organized list of layouts and widgets that can be dragged and dropped onto the required locations of your GUI. Other features for tinkering with the form can be accessed by right-clicking and opening up various context menus.

Another very useful dock widget is the `Property Editor` displayed in Figure 7-4. The properties of windows, widgets, and layouts such as an object's name, size constraints, status tips, and more can all be altered using the `Property Editor`. Each widget you add to a form will have its own set of properties as well as ones that the widget inherits from other classes. To select a specific widget, you can either click the object in the form or the widget's name in the `Object Inspector` dock widget.

The `Object Inspector` allows you to view all of the objects that are currently being used as well as their hierarchical layout. In Figure 7-5, you can see how the `MainWindow` is listed first, followed by the `centralwidget` and all of its widgets. If your form also has a menu or toolbar, then they will also be listed in the `Object Inspector` along with their corresponding actions.

NoteThe main layout for your GUI is not displayed in the `Object Inspector`. A broken layout icon (a red circle with a slash) is displayed on the central widget or on containers if no layout has been assigned to them.

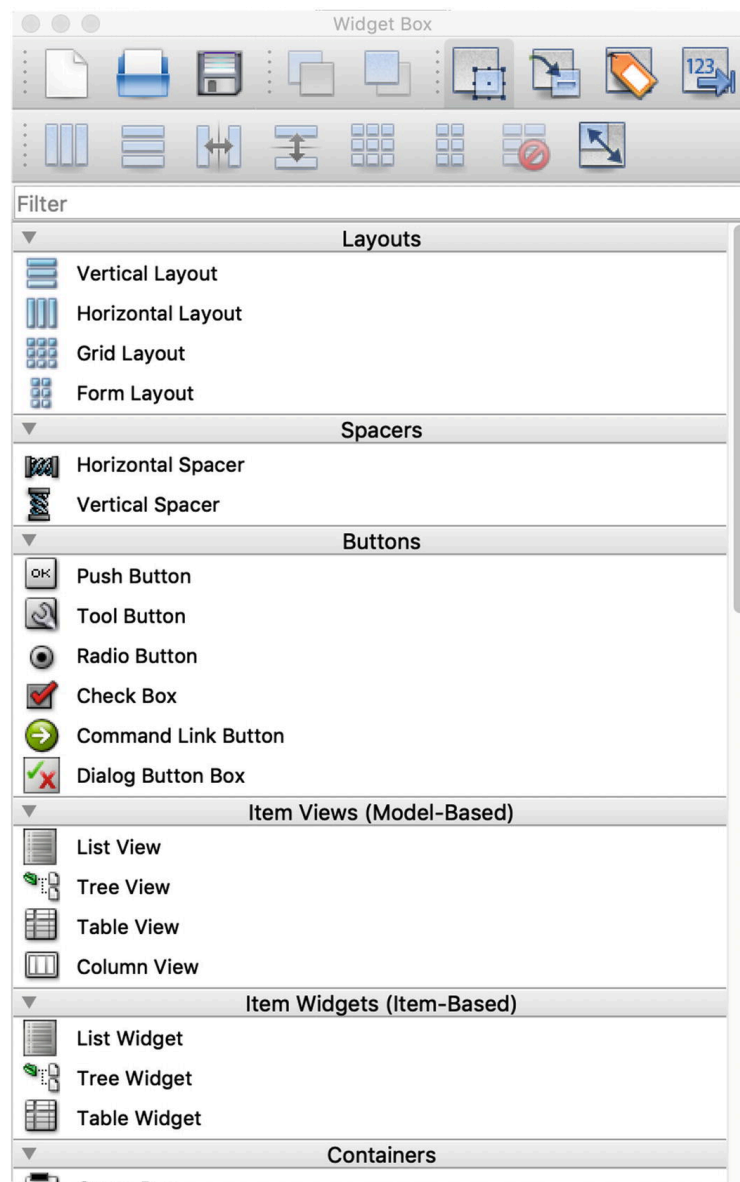


Figure 7-3 The Widget Box dock widget for selecting layouts and widgets

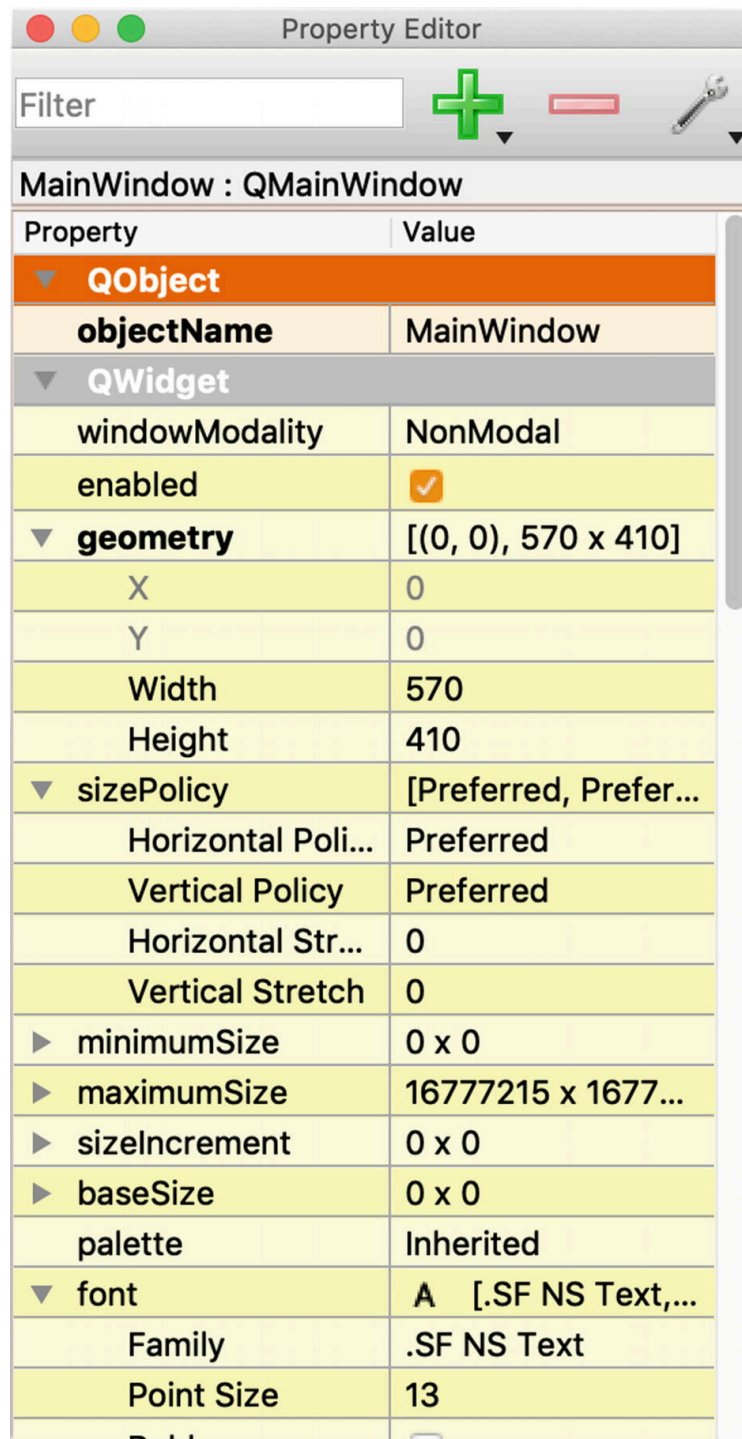


Figure 7-4 The Property Editor dock widget is used for setting the attributes of widgets

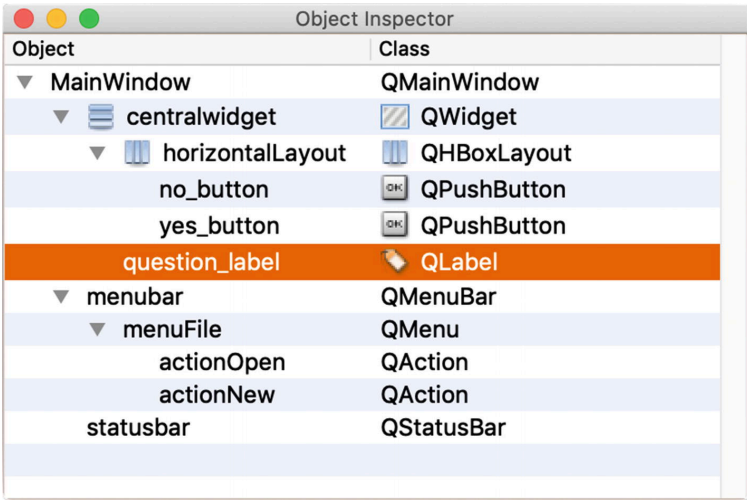


Figure 7-5 The Object Inspector displays the widget, layout, and menu objects

In Qt Designer, it is also possible to create, edit, and delete signals and slots between objects using the `Signal/Slot Editor`. You should be aware that although you can connect signals and slots, you will not always be able to completely configure your widgets and will sometimes need to complete that yourself later in the code. The `Signal/Slot Editor` can be seen in Figure 7-6. Qt Designer also provides an editing mode for connecting widgets.

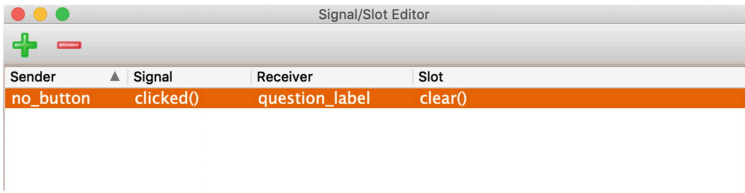


Figure 7-6 The Signal/Slot Editor for connecting the signals and slots of objects

Items in a menu, submenu, or a toolbar are assigned commands by using actions. These actions can then be given a shortcut key, made checkable, and more. The `Action Editor` seen in Figure 7-7 gives you access to working with actions. For more information about assigning actions, refer to Chapter 5.

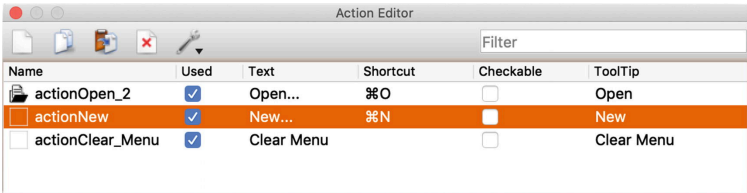


Figure 7-7 The Action Editor is used to manage the actions of menu items

Finally, there is the `Resource Browser` which allows you specify and manage resources you need to use in your application. These resources can include images and icons. The `Resource Browser` dock widget can be seen in Figure 7-8.

If you need to add resources, you first need to create a new resource file. To do so, click the pencil in the top-left corner of the `Resource Browser` dock widget. This will open an `Edit Resources` dialog similar to the one in Figure 7-9. Next, click the `Create New Resource` button, navigate to the correct directory, and enter a name for the resource file. The file will be saved with a `.qrc` file extension, which stands for Qt Resource Collection

and contains a list of all the resources used in your program. From here, create a prefix for managing the types of resources and begin adding files such as images and icons. When you are finished, click the OK button and the files will be added to the `Resource Browser`.

To access files in code found in the resource file, append `:/` to the beginning of the file's location. For example, to use the `new_file` icon

```
self.label.setPixmap(QtGui.QPixmap(":/icons/images/new_file.png"))
```

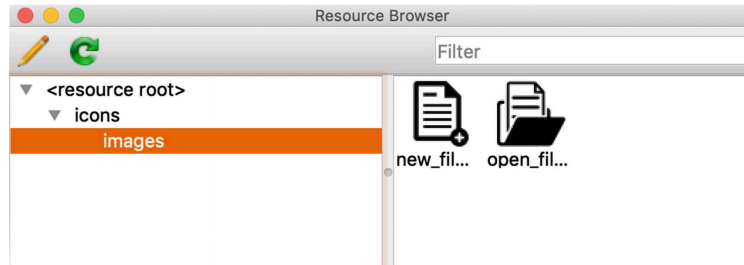


Figure 7-8 The Resource Browser for working with resources such as images and icons

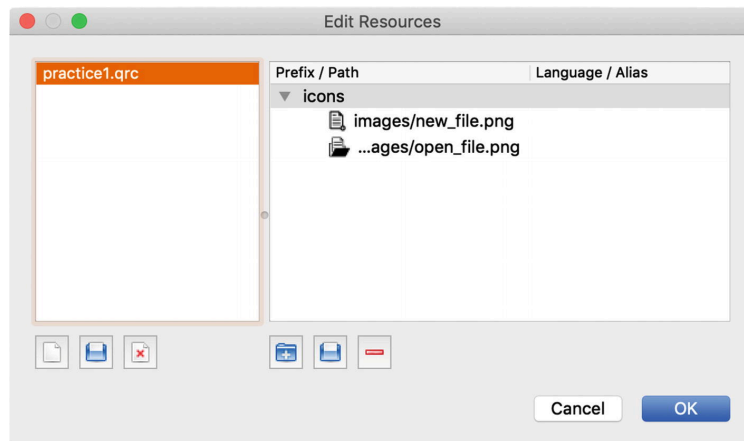


Figure 7-9 The Edit Resources dialog

Qt Designer's Editing Modes

In Qt Designer there are four different editing modes that can be accessed either in the Edit menu or from Qt Designer's toolbar. Take a look at [Figure 7-10](#) to help you locate the widgets in the toolbar.

1. `Edit Widgets` – Widgets can be dragged and dropped to a form, layouts can be applied, and objects can be edited both on the form and in the `Property Editor`. This is the default mode.
2. `Edit Signals/Slots` – Connect signals and slots for widgets and layouts. To create connections, click an object and drag the cursor toward an object that will receive the signal. Items that can be connected will be highlighted as the mouse cursor moves over them. To create the connection, release the mouse button once a line with an arrow connects the two objects. Then configure the signals and slots. Use in conjunction with the `Signal/Slots Editor` dock widget to edit connections.

3. **Edit Buddies** – Connect QLabel widgets with shortcuts to input widgets such as QLineEdit or QTextEdit. The input widget becomes the QLabel’s “buddy.” When the user enters the label’s shortcut key, the focus moves to the input widget.
4. **Edit Tab Order** – Set the order in which widgets receive focus when the `tab` key is pressed. This allows the user to navigate through the different widgets to make your application easier to use.



Figure 7-10 Qt Designer's Editing Modes (outlined in red). (From left to right) Edit Widgets, Edit Signals/Slots, Edit Buddies, Edit Tab Order

Creating an Application in Qt Designer

When you are creating your GUI's windows and widgets, you will probably continue to make slight adjustments to your application before it is finished. Fortunately, there are a few steps you can follow to simplify the building process.

1. **Select a form** – In the **New Form** dialog (shown in Figure 7-2), choose from one of the available templates, Main Window, Widget, or a type of dialog. You can also add and preview widgets to include in your GUI.
2. **Arrange objects on the form** – Use Qt Designer's drag and drop mechanics to place widgets on the form. Then assign layouts to containers and the main window.
3. **Edit the properties of objects** – Click the objects in the form and edit their features in the **Property Editor** dock widget.
4. **Connect signals and slots** – Use the **Signal/Slots** Editing mode to link signals to slots.
5. **Preview your GUI** – Examine the form before saving it as a UI file with the `.ui` extension.
6. **Create and edit Python code** – Utilize the `pyuic` compiler to convert the UI file to readable and editable Python code.

The following project will cover these steps in addition to many of the basic concepts for creating GUIs using Qt Designer.

Project 7.1 – Keypad GUI

To get you started using Qt Designer, the project in this chapter is a simple one – a keypad GUI. A keypad is a set of buttons with digits, symbols, or letters used as an input device for passcodes, telephone numbers, and more. They can be found on a number of devices such as calculators, cell phones, and locks. Figure 7-11 shows the keypad GUI you will create in this project.

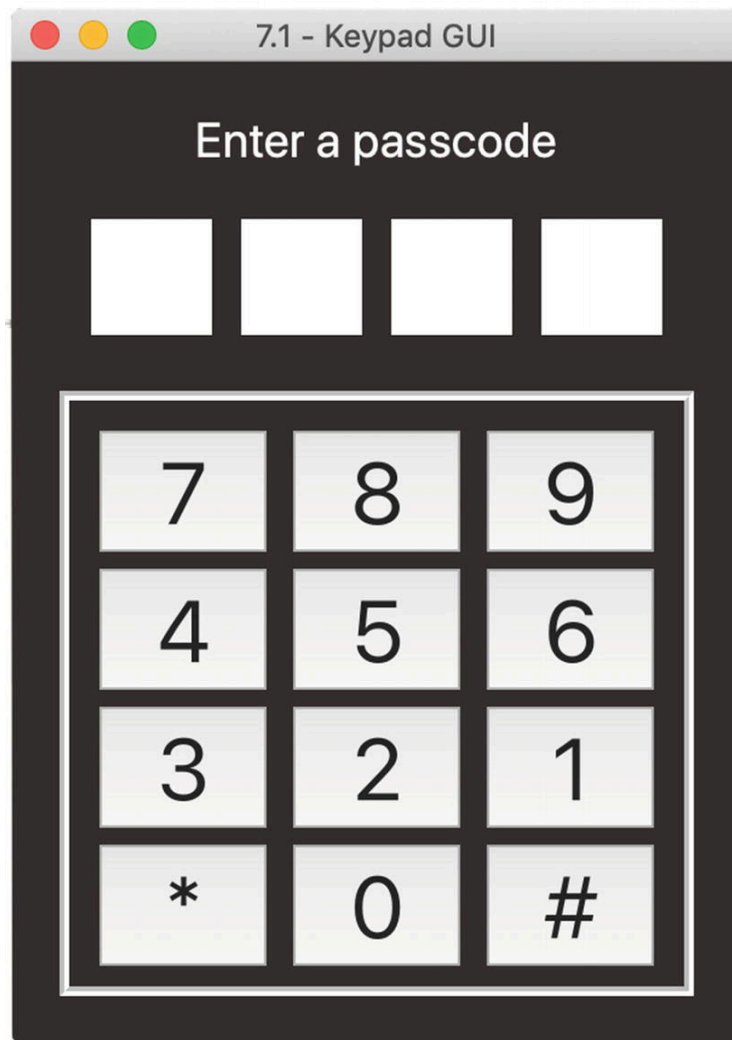


Figure 7-11 Keypad GUI

Keypad GUI Solution

The keypad application is comprised of two Python files, `keypad_gui.py` and `keypad_main.py`. The `keypad_gui.py` contains all of the Python code generated from the UI file created using Qt Designer. In order to use that code, we then create a customized class in a separate file, `keypad_main.py`, to import and set up the GUI.

The keypad GUI consists of four `QLineEdit` widgets to input only numeric values, twelve `QPushButton` widgets, and a single `QLabel` to display information about how to use the interface. The asterisk button allows users to clear the current input and the hash button is for confirming the user's four-digit input.

This project introduces the `QFrame` container for organizing Qt widgets. The program also utilizes nested layouts to arrange the various widgets and containers.

Note The following Python code in Listing 7-1 is produced from the UI file using `pyuic`. It has not been altered. To help you understand the code, Listing 7-1 is broken into parts with annotations.

```
# keypad_gui.py
# Import necessary modules
from PyQt5 import QtCore, QtGui, QtWidgets
class Ui_Keypad(object):
    def setupUi(self, Keypad):
        Keypad.setObjectName("Keypad")
        Keypad.resize(302, 406)
```

Listing 7-1 Code for keypad created from keypad.ui

Import modules from PyQt5 and create a class that inherits from `QWidget`, `Keypad`. The member function `setupUi()` of the class `Ui_Keypad` is used to build a widget tree on the parent widget, `Keypad`. A **widget tree** is used to represent the organization of widgets in a UI. So the `setupUi()` method composes the UI based upon the widgets and connections we used to create it along with the parameters it inherits from its parent widget.

```
palette = QtGui.QPalette()
brush = QtGui.QBrush(QtGui.QColor(255, 255, 255))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Active, QtGui.QPalette.Base, brush)
brush = QtGui.QBrush(QtGui.QColor(52, 48, 47))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Active, QtGui.QPalette.Window, brush)
brush = QtGui.QBrush(QtGui.QColor(255, 255, 255))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Inactive, QtGui.QPalette.Base, brush)
brush = QtGui.QBrush(QtGui.QColor(52, 48, 47))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Inactive, QtGui.QPalette.Window, brush)
brush = QtGui.QBrush(QtGui.QColor(52, 48, 47))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Disabled, QtGui.QPalette.Base, brush)
brush = QtGui.QBrush(QtGui.QColor(52, 48, 47))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Disabled, QtGui.QPalette.Window, brush)
Keypad.setPalette(palette)
```

Every widget in Qt has a palette which contains information about how they will be drawn in the window. The **QPalette** class contains the color groups for each widget during one of three possible states – `Active`, `Inactive`, or `Disabled`. The preceding code changes the color of the main window to dark gray. How to change the settings in the palette will be introduced in the “Explanation” section of Project 7.1. You can also create style sheets in Qt Designer.

```
self.verticalLayout = QtWidgets.QVBoxLayout(Keypad)
self.verticalLayout.setObjectName("verticalLayout")
```

Create the vertical layout that will be used for the main window.

```
self.label = QtWidgets.QLabel(Keypad)
palette = QtGui.QPalette()
brush = QtGui.QBrush(QtGui.QColor(255, 255, 255))
brush.setStyle(QtCore.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Active, QtGui.QPalette.WindowText, brush)
brush = QtGui.QBrush(QtGui.QColor(255, 255, 255))
brush.setStyle(QtCore.Qt.SolidPattern)
```

```
palette.setBrush(QtGui.QPalette.Inactive, QtGui.QPalette.WindowText, brush)
brush = QtGui.QBrush(QtGui.QColor(127, 127, 127))
brush.setStyle(QtGui.Qt.SolidPattern)
palette.setBrush(QtGui.QPalette.Disabled, QtGui.QPalette.WindowText, brush)
self.label.setPalette(palette)
font = QtGui.QFont()
font.setPointSize(20)
self.label.setFont(font)
self.label.setAlignment(QtGui.Qt.AlignCenter)
self.label.setObjectName("label")
self.verticalLayout.addWidget(self.label)
```

Create the QLabel object, modify its palette settings so that the color of the font is light gray, and add the label to the vertical layout.

```
self.frame = QtWidgets.QFrame(Keypad)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Preferred, QtWidgets.QSizePolicy.Preferred)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(1)
sizePolicy.setHeightForWidth(self.frame.sizePolicy().hasHeightForWidth())
self.frame.setSizePolicy(sizePolicy)
self.frame.setFrameShape(QtWidgets.QFrame.NoFrame)
self.frame.setFrameShadow(QtWidgets.QFrame.Plain)
self.frame.setLineWidth(0)
self.frame.setObjectName("frame")
self.horizontalLayout = QtWidgets.QHBoxLayout(self.frame)
self.horizontalLayout.setObjectName("horizontalLayout")
```

The first QFrame container, frame, will hold four QLineEdit widgets and use a horizontal layout. You can adjust the style of a frame object using its setFrameShape(), setFrameShadow(), and other methods.

```
self.line_edit1 = QtWidgets.QLineEdit(self.frame)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.line_edit1.sizePolicy().hasHeightForWidth())
self.line_edit1.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(30)
self.line_edit1.setFont(font)
self.line_edit1.setAlignment(QtGui.Qt.AlignCenter)
self.line_edit1.setObjectName("line_edit1")
self.horizontalLayout.addWidget(self.line_edit1)
self.line_edit2 = QtWidgets.QLineEdit(self.frame)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.line_edit2.sizePolicy().hasHeightForWidth())
self.line_edit2.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(30)
self.line_edit2.setFont(font)
self.line_edit2.setAlignment(QtGui.Qt.AlignCenter)

self.line_edit2.setObjectName("line_edit2")
self.horizontalLayout.addWidget(self.line_edit2)
self.line_edit3 = QtWidgets.QLineEdit(self.frame)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.QSizePolicy.Expanding)
```

```

sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.line_edit3.sizePolicy().hasHeightForWidth())
self.line_edit3.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(30)
self.line_edit3.setFont(font)
self.line_edit3.setAlignment(QtCore.Qt.AlignCenter)
self.line_edit3.setObjectName("line_edit3")
self.horizontalLayout.addWidget(self.line_edit3)
self.line_edit4 = QtWidgets.QLineEdit(self.frame)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Expanding, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.line_edit4.sizePolicy().hasHeightForWidth())
self.line_edit4.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(30)
self.line_edit4.setFont(font)
self.line_edit4.setAlignment(QtCore.Qt.AlignCenter)
self.line_edit4.setObjectName("line_edit4")
self.horizontalLayout.addWidget(self.line_edit4)
self.verticalLayout.addWidget(self.frame)

```

Each of the four line edit widgets has size policies that allow them to stretch if the window resizes in both the vertical and horizontal directions by using `QSizePolicy.Expanding`. They are then arranged in the `horizontalLayout` of the `frame` container. The `frame` object is then added to the `verticalLayout` of the main window.

```

self.frame_2 = QtWidgets.QFrame(Keypad)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Preferred, QtWidgets.QSizePolicy.Preferred)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(2)
sizePolicy.setHeightForWidth(self.frame_2.sizePolicy().hasHeightForWidth())
self.frame_2.setSizePolicy(sizePolicy)
self.frame_2.setFrameShape(QtWidgets.QFrame.Box)
self.frame_2.setFrameShadow(QtWidgets.QFrame.Sunken)
self.frame_2.setLineWidth(2)
self.frame_2.setObjectName("frame_2")
self.gridLayout = QtWidgets.QGridLayout(self.frame_2)
self.gridLayout.setObjectName("gridLayout")

```

Instantiate the second frame container, and set the size policy and style attributes. The layout inside `frame_2` houses the twelve buttons and uses a grid layout.

```

self.button_7 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_7.sizePolicy().hasHeightForWidth())
self.button_7.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_7.setFont(font)
self.button_7.setObjectName("button_7")
self.gridLayout.addWidget(self.button_7, 0, 0, 1, 1)
self.button_8 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)

```

```

sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_8.sizePolicy().hasHeightForWidth())
self.button_8.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_8.setFont(font)
self.button_8.setObjectName("button_8")
self.gridLayout.addWidget(self.button_8, 0, 1, 1, 1)
self.button_9 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_9.sizePolicy().hasHeightForWidth())
self.button_9.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_9.setFont(font)
self.button_9.setObjectName("button_9")
self.gridLayout.addWidget(self.button_9, 0, 2, 1, 1)
self.button_4 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_4.sizePolicy().hasHeightForWidth())
self.button_4.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)

self.button_4.setFont(font)
self.button_4.setObjectName("button_4")
self.gridLayout.addWidget(self.button_4, 1, 0, 1, 1)
self.button_5 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_5.sizePolicy().hasHeightForWidth())
self.button_5.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_5.setFont(font)
self.button_5.setObjectName("button_5")
self.gridLayout.addWidget(self.button_5, 1, 1, 1, 1)
self.button_6 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)

sizePolicy.setHeightForWidth(self.button_6.sizePolicy().hasHeightForWidth())
self.button_6.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_6.setFont(font)
self.button_6.setObjectName("button_6")
self.gridLayout.addWidget(self.button_6, 1, 2, 1, 1)
self.button_3 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)

```

```

sizePolicy.setHeightForWidth(self.button_3.sizePolicy().hasHeightForWidth())
self.button_3.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_3.setFont(font)
self.button_3.setObjectName("button_3")
self.gridLayout.addWidget(self.button_3, 2, 0, 1, 1)
self.button_2 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_2.sizePolicy().hasHeightForWidth())
self.button_2.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_2.setFont(font)
self.button_2.setObjectName("button_2")
self.gridLayout.addWidget(self.button_2, 2, 1, 1, 1)
self.button_1 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_1.sizePolicy().hasHeightForWidth())
self.button_1.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_1.setFont(font)
self.button_1.setObjectName("button_1")
self.gridLayout.addWidget(self.button_1, 2, 2, 1, 1)
self.button_star = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_star.sizePolicy().hasHeightForWidth())
self.button_star.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_star.setFont(font)
self.button_star.setObjectName("button_star")
self.gridLayout.addWidget(self.button_star, 3, 0, 1, 1)
self.button_0 = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.Expanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_0.sizePolicy().hasHeightForWidth())
self.button_0.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_0.setFont(font)
self.button_0.setObjectName("button_0")
self.gridLayout.addWidget(self.button_0, 3, 1, 1, 1)
self.button_hash = QtWidgets.QPushButton(self.frame_2)
sizePolicy = QtWidgets.QSizePolicy(QtWidgets.QSizePolicy.Minimum, QtWidgets.QSizePolicy.MinimumExpanding)
sizePolicy.setHorizontalStretch(0)
sizePolicy.setVerticalStretch(0)
sizePolicy.setHeightForWidth(self.button_hash.sizePolicy().hasHeightForWidth())
self.button_hash.setSizePolicy(sizePolicy)
font = QtGui.QFont()
font.setPointSize(36)
self.button_hash.setFont(font)
self.button_hash.setObjectName("button_hash")

```

```
self.gridLayout.addWidget(self.button_hash, 3, 2, 1, 1)
self.verticalLayout.addWidget(self.frame_2)
```

The twelve `QPushButton` widgets are created, and their properties, such as their object names and font sizes, are adjusted. Every button is then added to the grid layout of `frame_2` which is then added to the vertical layout of the main window.

```
self.retranslateUi(Keypad)
self.button_star.clicked.connect(self.line_edit1.clear)
self.button_star.clicked.connect(self.line_edit2.clear)
self.button_star.clicked.connect(self.line_edit3.clear)
self.button_star.clicked.connect(self.line_edit4.clear)
QtCore.QMetaObject.connectSlotsByName(Keypad)
```

The `retranslateUi()` method handles how to display text in the GUI in the situation where a different language is used. In the keypad, the user is given a way to delete their input and try again. When the `button_star` is clicked, it sends a signal to clear the text in all four `QLineEdit` widgets. This could be handled a different way, but for this example, this is used as an example to show how to connect signals to slots in Qt Designer.

```
def retranslateUi(self, Keypad):
    _translate = QtCore.QCoreApplication.translate
    Keypad.setWindowTitle(_translate("Keypad", "7.1 - Keypad GUI"))
    self.label.setText(_translate("Keypad", "Enter a passcode"))
    self.button_7.setText(_translate("Keypad", "7"))
    self.button_8.setText(_translate("Keypad", "8"))
    self.button_9.setText(_translate("Keypad", "9"))
    self.button_4.setText(_translate("Keypad", "4"))
    self.button_5.setText(_translate("Keypad", "5"))
    self.button_6.setText(_translate("Keypad", "6"))
    self.button_3.setText(_translate("Keypad", "3"))
    self.button_2.setText(_translate("Keypad", "2"))
    self.button_1.setText(_translate("Keypad", "1"))
    self.button_star.setText(_translate("Keypad", "*"))
    self.button_0.setText(_translate("Keypad", "0"))
    self.button_hash.setText(_translate("Keypad", "#"))
```

The following code in Listing 7-2 creates the class that inherits from `Ui_Keypad` and sets up the GUI application.

```
# keypad_main.py
# Import necessary modules
import sys
from PyQt5 import QtCore, QtGui, QtWidgets
from PyQt5.QtWidgets import QMessageBox
from PyQt5.QtGui import QIntValidator
from keypad_gui import Ui_Keypad
class KeypadGUI(QtWidgets.QWidget):
    def __init__(self):
        super(KeypadGUI, self).__init__()
        self.ui = Ui_Keypad()
        self.ui.setupUi(self)
        self.initializeUI()
        self.show()
    def initializeUI(self):
        # Update other line_edit features
        self.ui.line_edit1.setMaxLength(1) # Set the max number of characters allowed
```

```

self.ui.line_edit1.setValidator(QIntValidator(0, 9)) # User can only enter ints from 0-9
self.ui.line_edit1.setFocusPolicy(QtCore.Qt.NoFocus) # Widget does not except focus
self.ui.line_edit2.setMaxLength(1)
self.ui.line_edit2.setValidator(QIntValidator(0, 9))
self.ui.line_edit2.setFocusPolicy(QtCore.Qt.NoFocus)
self.ui.line_edit3.setMaxLength(1)
self.ui.line_edit3.setValidator(QIntValidator(0, 9))
self.ui.line_edit3.setFocusPolicy(QtCore.Qt.NoFocus)
self.ui.line_edit4.setMaxLength(1)
self.ui.line_edit4.setValidator(QIntValidator(0, 9))
self.ui.line_edit4.setFocusPolicy(QtCore.Qt.NoFocus)
# 4-digit passcode

self.passcode = 8618
#### Add signal/slot connections for buttons ####
self.ui.button_0.clicked.connect(lambda: self.numberClicked(self.ui.button_0.text()))
self.ui.button_1.clicked.connect(lambda: self.numberClicked(self.ui.button_1.text()))
self.ui.button_2.clicked.connect(lambda: self.numberClicked(self.ui.button_2.text()))
self.ui.button_3.clicked.connect(lambda: self.numberClicked(self.ui.button_3.text()))
self.ui.button_4.clicked.connect(lambda: self.numberClicked(self.ui.button_4.text()))
self.ui.button_5.clicked.connect(lambda: self.numberClicked(self.ui.button_5.text()))
self.ui.button_6.clicked.connect(lambda: self.numberClicked(self.ui.button_6.text()))
self.ui.button_7.clicked.connect(lambda: self.numberClicked(self.ui.button_7.text()))
self.ui.button_8.clicked.connect(lambda: self.numberClicked(self.ui.button_8.text()))
self.ui.button_9.clicked.connect(lambda: self.numberClicked(self.ui.button_9.text()))
self.ui.button_hash.clicked.connect(self.checkPasscode)
def numberClicked(self, text_value):
    """
    When a button with a digit is pressed, check if the text for QLineEdit widgets
    are empty. If empty, set the focus to the correct widget and enter text value

    """
    if self.ui.line_edit1.text() == "":
        self.ui.line_edit1.setFocus()
        self.ui.line_edit1.setText(text_value)
        self.ui.line_edit1.repaint()
    elif (self.ui.line_edit1.text() != "") and (self.ui.line_edit2.text() == ""):
        self.ui.line_edit2.setFocus()
        self.ui.line_edit2.setText(text_value)
        self.ui.line_edit2.repaint()
    elif (self.ui.line_edit1.text() != "") and (self.ui.line_edit2.text() != "") \
        and (self.ui.line_edit3.text() == ""):
        self.ui.line_edit3.setFocus()
        self.ui.line_edit3.setText(text_value)
        self.ui.line_edit3.repaint()
    elif (self.ui.line_edit1.text() != "") and (self.ui.line_edit2.text() != "") \
        and (self.ui.line_edit3.text() != "") and (self.ui.line_edit4.text() == ""):
        self.ui.line_edit4.setFocus()
        self.ui.line_edit4.setText(text_value)
        self.ui.line_edit4.repaint()
def checkPasscode(self):
    """
    Concatenate the text values from the 4 QLineEdit widgets, and check to see if the passcode ente
    """
    entered_passcode = self.ui.line_edit1.text() + self.ui.line_edit2.text() + \
        self.ui.line_edit3.text() + self.ui.line_edit4.text()
    if len(entered_passcode) == 4 and int(entered_passcode) == self.passcode:
        QMessageBox.information(self, "Valid Passcode!", "Valid Passcode!", QMessageBox.Ok, QMessag

```

```

        self.close()
    else:
        QMessageBox.warning(self, "Error Message", "Invalid Passcode.", QMessageBox.Close, QMessageBox.Ok)
        self.ui.line_edit1.clear()
        self.ui.line_edit2.clear()
        self.ui.line_edit3.clear()
        self.ui.line_edit4.clear()
        self.ui.line_edit1.setFocus()
if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    Keypad = KeypadGUI()
    sys.exit(app.exec_())

```

Listing 7-2 Code for the keypad GUI created from keypad.ui

When you run the code, your GUI should look similar to Figure [7-11](#).

Explanation

In order to utilize the `Ui_Keypad` class that was created using Qt Designer, we create a new Python file, `keypad_main.py`. The `KeypadGUI` class created in `keypad_main.py` will inherit from the `Ui_Keypad` class.

We begin by importing the modules needed for this project, including the `Ui_Keypad` class and a new Qt class, `QIntValidator`. Qt provides a few classes that can be used to verify the types of input text. `QIntValidator` will be used to check if the values input into the `QLineEdit` widgets are integers.

The `KeypadGUI` class is created using a **single inheritance** approach where it inherits its properties from a single parent class, `QWidget`. The user interface is set up in the `__init__()` method in the following lines:

```

self.ui = Ui_Keypad()
self.ui.setupUi(self)

```

In the `initializeUI()` method, local modifications are made to the `QLineEdit` widgets. Here the line edit widget's focus policy is set to `NoFocus` so that users can only enter input in the correct order.

Then we connect the signals and slots for the button widgets. When each button is clicked, it sends a signal that is connected to the `numberClicked()` slot. Rather than creating a separate method for each button, the `lambda` function is used to reuse a method for signals. `lambda` calls the `numberClicked()` function and passes it a new parameter every time, in this case the specific text from each button.

When a user clicks a button, that button's number needs to appear in the correct line edit widget from left to right. A widget receives focus if its `text()` value is empty. On MacOS, the `repaint()` method was used to update the text in the `QLineEdit` widgets. `repaint()` is used to force a widget to update itself.

Finally, if the user presses the # button, the method `checkPasscode()` checks if the user entered the passcode that matches `self.passcode`. If the input does not match, the line edit widgets are reset. This project

could be designed so that the password is read from a file or from a database.

We have taken a look at the user interface class created from Qt Designer and at the file that inherits from it. The following section shows in detail how to create the GUI in Qt Designer.

Select a Form

Begin by opening up Qt Designer. Choose the `Widget` template from `New Form` dialog box. We will use the default screen size. Select `Create`. This opens up a blank GUI window with a grid of dots inside of the Qt Designer interface like in Figure 7-12.

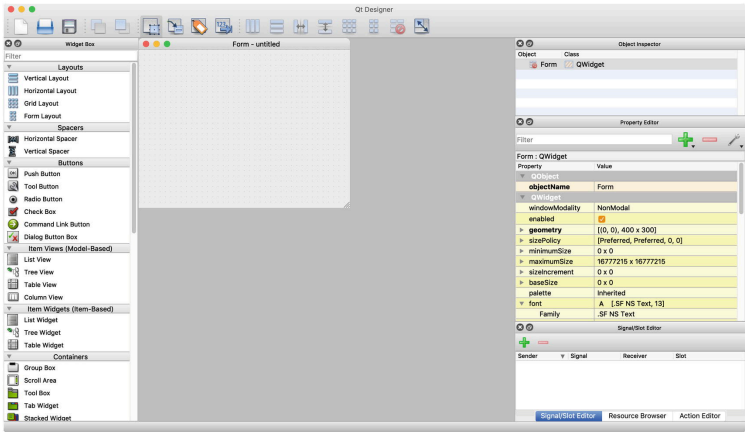


Figure 7-12 The Qt Designer interface displaying the toolbar and its different dock widgets for managing GUIs. In the center is the empty `Widget` form that will be used to create the keypad

Arrange Objects on the Form

From here you could immediately begin to adjust certain features of the form such as the window size or the background color. Instead, let’s begin by adding whatever widgets you may need for your project by dragging and dropping them into the main window from the `Widget Box` on the left of the window.

Start by selecting a `QLabel` widget and two `QFrame` containers and place them on the form like in Figure 7-13. You can resize the frames by clicking them and moving the edges of the frame. Then drag four `QLineEdit` widgets and arrange them in the top `QFrame` container . They will overlap, but that will be fixed when you apply layouts to the frames and the main window. When an object is dragged on top of a container that it can be placed into, the container will be highlighted to indicate that you can drop the widget inside. In addition, place twelve `QPushButton` widgets in the bottom frame.

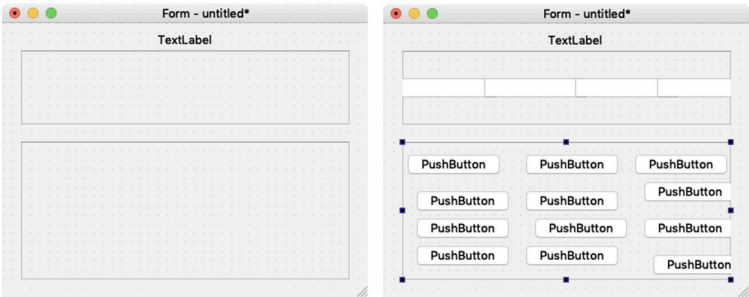


Figure 7-13 The form with a label and two frames (left) and with the line edit widgets and push buttons added (right)

The QFrame Class

The **QFrame** class is used as a container to group and surround widgets, or to act as placeholders in GUI applications. You can also apply a frame style to a **QFrame** container to visually separate it from nearby widgets. The following bit of code shows an example of how to create a frame object, modify its properties, and add a widget:

```
def frameUI(self):
    self.frame = QFrame(self) # Create QFrame object
    size_policy = QSizePolicy(QSizePolicy.Expanding, QSizePolicy.Preferred)
    self.frame.setSizePolicy(size_policy)
    self.frame.setFrameShape(QFrame.Box)
    self.frame.setFrameShadow(QFrame.Raised)
    self.frame.setLineWidth(3)
    self.frame.setMidLineWidth(5)
    # Set layout for QFrame object
    self.grid = QGridLayout(self.frame)
    # Place other widgets inside the frame by
    # calling the addWidget() method on the layout.
    self.button = QPushButton("Enter", self)
    self.grid.addWidget(self.button, 0, 0, 1, 0)
```

A frame object can have a number of different styles of frames, including boxes, panels, or lines. The style of the frame can be adjusted using the `setFrameShadow()`, `setLineWidth()`, and `setMidLineWidth()` methods.

Apply Layouts in Qt Designer

The next step is to add layouts to all of the containers and to the main window, as well. This is important to make sure that items are placed and resized correctly. Layouts can be added either from the toolbar or from context menus. It is possible to add more widgets to existing layouts once they have been set.

Since Qt Designer uses a drag and drop interface, you only need to place the objects on the form close to where you want them to be and then select one of the four layouts – `QGridLayout`, `QHBoxLayout`, `QVBoxLayout`, or `QFormLayout` – and Qt Designer will take care of placing them by using a widget's size hint. For more information about the types of layouts in PyQt, refer to Chapter 4.

Right-click the topmost frame to open a context menu. Scroll down to the last option, `Layout`, and select `Layout Out Horizontally`. Do the same thing for the bottom frame, but this time select `Layout Out in a Grid`. This is demonstrated in Figure 7-14.

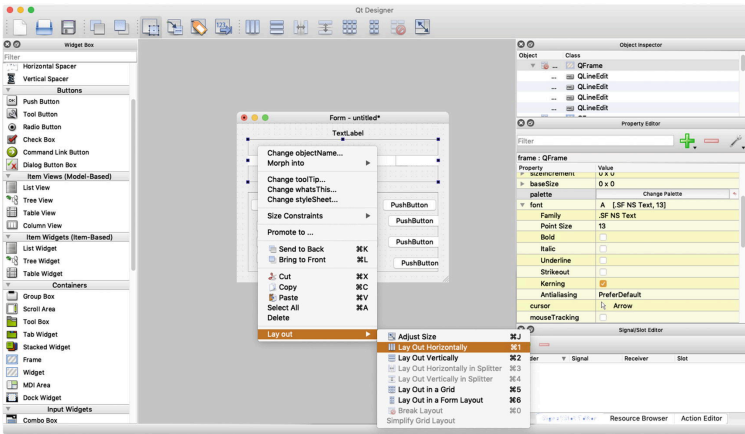


Figure 7-14 Open a context menu to select layouts for containers and windows

The top-level layout of a form can be set by right-clicking the form itself in the main window, and locating the layout you want to use. For the keypad GUI, select `Layout Out Vertically`. If the widgets are not aligned properly, you can also open the context menu, select `Break Layout`, and rearrange the widgets. Figure 7-15 shows the form with layouts applied.

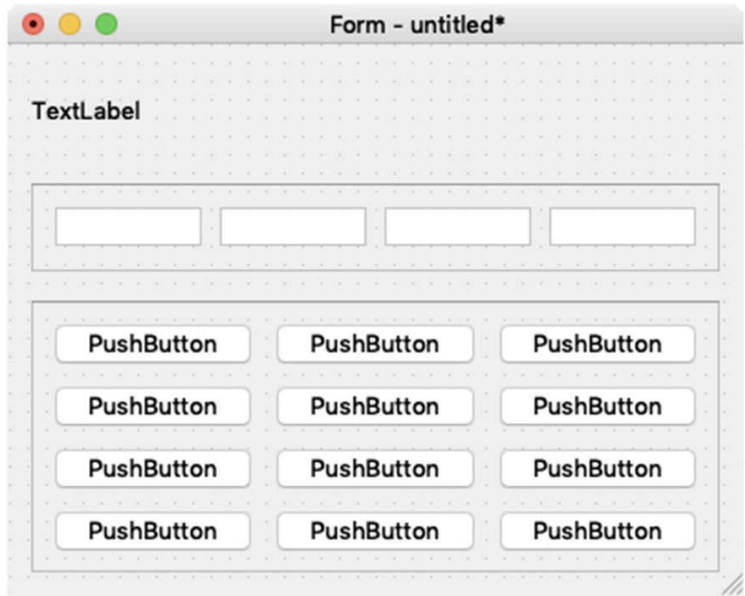


Figure 7-15 The keypad GUI with layouts

Edit the Properties of Objects

Once you have the layouts prepared, you should begin editing the features of the objects. This step could also be accomplished earlier when you place objects on the form.

The `Property Editor` is shown in Figure 7-4. It is organized into two columns, `Property` and `Value`. The properties are organized by Qt Classes.

To access and make changes to specific containers, widgets, layouts, or even the main window, you can click them in the form or in the `Object Inspector`. If a property is edited in the `Property Editor`, you can locate it by the following pattern:

Qt Class (Property column) ► Property name ► (submenu, if any) ► Value

The following are the steps that you can follow along to create the keypad GUI in Qt Designer:

1. Change window title: `QWidget ► windowTitle ► 7.1 ► Keypad GUI`
2. Double-click the `QLabel`. Change text to `Enter a passcode.`
3. Change `QLabel` properties:
 - a. `QWidget ► font ► font ► Point Size ► 20`
 - b. To edit palette colors, you will need to locate the `palette` property that opens a dialog box. Here you can change the colors for different parts of an object. To change the color of the text in the label object: `QWidget ► palette ► Change Palette ► Window Text ► white`
 - c. `QLabel ► alignment ► Horizontal ► AlignHCenter`
4. Change top frame properties:
 - a. `QWidget ► sizePolicy ► Vertical Stretch ► 1`
 - b. `QFrame ► frameShape ► NoFrame`
 - c. `QFrame ► frameShadow ► Plain`
5. For each of the four `QLineEdit` widgets, modify their properties:
 - a. `QWidget ► sizePolicy ► Vertical Policy ► Expanding`
 - b. `QWidget ► font ► font ► Point Size ► 30`
 - c. `QLineEdit ► alignment ► Horizontal ► AlignHCenter`
6. Change bottom frame properties:
 - a. `QWidget ► sizePolicy ► Vertical Stretch ► 2`
 - b. `QFrame ► frameShape ► Box`
 - c. `QFrame ► frameShadow ► Sunken`
 - d. `QFrame ► lineWidth ► 2`
7. Double-click each of the buttons and change their text to `0-9, *,` and `#.`
8. Edit each of the buttons' properties:
 - a. `QWidget ► sizePolicy ► Vertical Policy ► Expanding`
 - b. `QWidget ► font ► font ► Point Size ► 36`
9. Resize the main window:
 - a. `QWidget ► geometry ► Width ► 302`
 - b. `QWidget ► geometry ► Height ► 406`
10. Click the form and change its background color: `QWidget ► palette ► Change Palette ► Window ► dark gray`
11. In the `Object Inspector`, double-click each of the default object names and edit them. The object name is used to reference the objects in the code.

After you have followed along with each of the steps, the form should look similar to Figure [7-11](#).

Connect Signals and Slots in Qt Designer

Switch to the `Edit Signals/Slots` mode by selecting it from the toolbar. Qt Designer has a simple interface for connecting signals and slots. Click the object that will emit a signal and drag it to the object that will receive the signal, which is the slot.

For the keypad GUI, we are only making one set of connections. The remaining signals and slots will be handled by manually coding them. When the '*' button is clicked, we want to clear all four line edit widgets. Click the button and drag the red arrow to the first line edit object. A dialog box will appear (displayed in Figure 7-16) that allows you to select the methods for both the signal and the slot. Select `clicked()` for the button and `clear()` for the line edit. Finish connecting the other three line edit widgets. Refer to Figure 7-17 as a guide for connecting the widgets.

Tip When connecting signals and slots, make sure to check the “Show signals and slots inherited from QWidget” checkbox to access more methods.

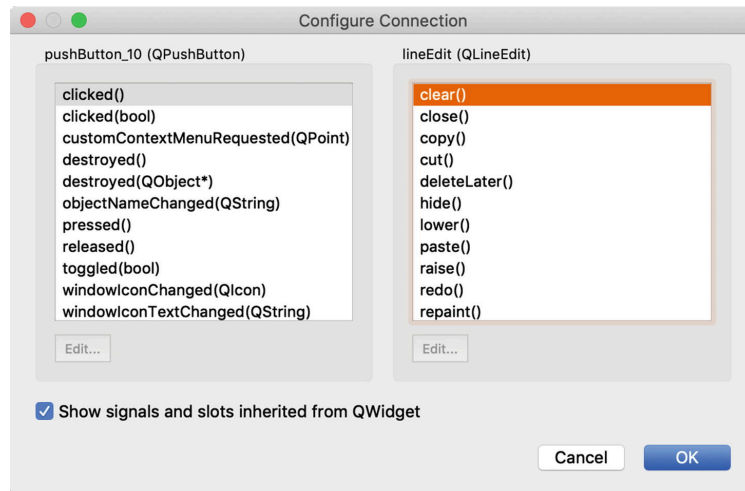


Figure 7-16 The dialog box for connecting signals and slots

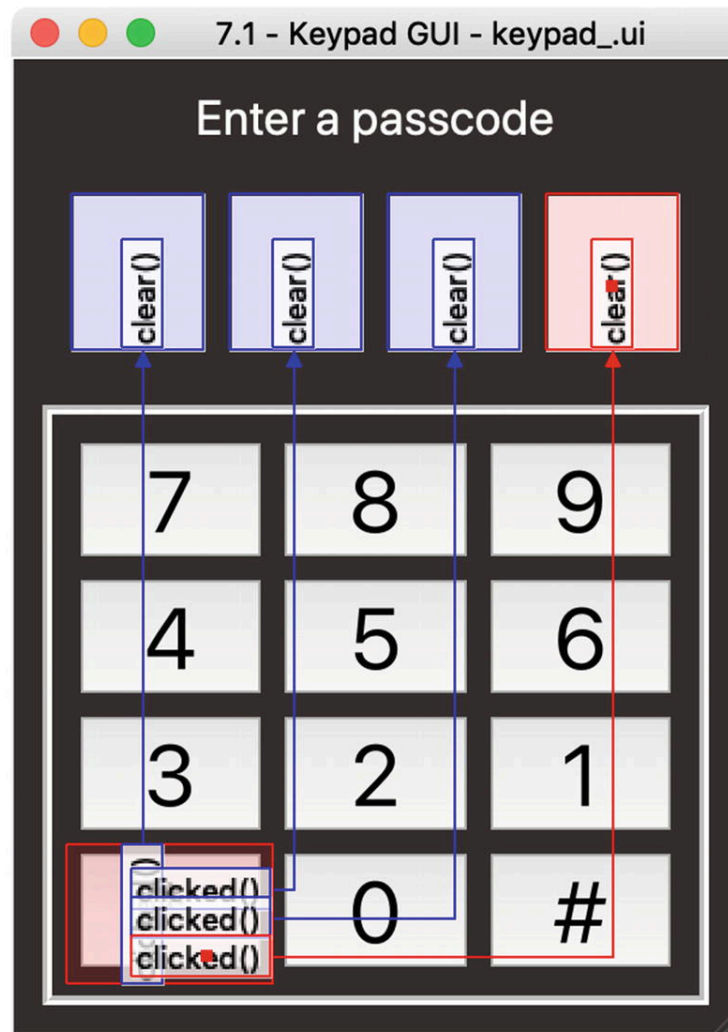


Figure 7-17 The keypad GUI with signal and slot connections

Preview Your GUI

It is often useful to view and interact with the form before exporting it to code. Not only can this be useful for checking the visual appearance of your GUI, but previewing also helps to make sure the signals and slots, re-sizing the window, and other functions are working properly.

To preview a form, open the **Form** menu and select **Preview** or use the hot keys **Ctrl+R** for Windows or **Cmd+R** for MacOS. If you are satisfied with your form, save it as a UI file with the `.ui` extension. Qt Designer UI files are written in XML format and contain the widget tree representation for creating a GUI.

Create and Edit Python Code

Qt Designer uses the Qt utility User Interface Compiler (`uic`) to generate code and create the user interface. However, since you are using PyQt5, you must use the `uic` Python module, `pyuic5`, to load `.ui` files and convert the XML file to Python code.

The `pyuic5` utility is a command-line interface for interacting with `uic`. Open up the command prompt in Windows or Terminal in MacOS and navigate to the directory that contains the UI file. The format for converting to Python code is

```
pyuic5 filename.ui -o filename.py
```

To output a Python file, you need to include the `-o` option and the Python file to be written to, `filename.py`. This command will generate a single Python class. Generally, you will need to create a separate file to inherit from your newly created user interface class. Another option is to create an executable file that can display the GUI. This can be done by including the `-x` option.

```
pyuic5 -x filename.ui -o filename.py
```

NoteIf you make changes to the GUI in Qt Designer after creating the Python file, you will need to call `pyuic5` again on the UI file.

TipWhile it is possible to write new code in the newly generated Python file, the best thing to do would be to create a new file that imports the new user interface class. If you need to make changes to the GUI in Qt Designer and resave the file, it will erase any new code that you have written.

Extra Tips for Using Qt Designer

The following section briefly covers two additional topics:

- Creating GUIs with menus
- Displaying images in Qt Designer

Setting Up Main Windows and Menus

Open Qt Designer and select the `Main Window` template from the `Form Menu` in Figure 7-2. This creates a main window with a menubar and status bar by default. You can see a main window form displayed in Figure 7-1.

Adding Menus and Submenus in Qt Designer

Adding menus in Qt Designer is simple. Double-click the `Type Here` placeholder text in the menubar and enter the `title` of the menu. This process is shown in Figure 7-18. If you want to create a shortcut, you can also add the ampersand, `'&'`, to the beginning of the menu's text. This updates the `menubar` object in the `Object Inspector` dialog. You can also edit the menu's properties in the `Property Editor`.

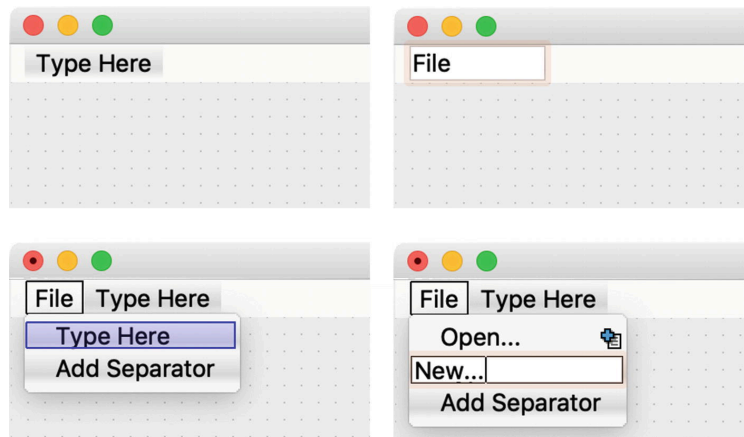


Figure 7-18 Creating menus and menu entries. Type Here placeholder (top-left). Double-click the placeholder and enter menu's title (top-right). Add new menu entry (bottom-left). New menu entry (bottom-right)

From here you can either add more menus, submenus, or actions. To add a submenu, first create a menu item. Then click the plus symbol next to the new entry in the menu. This will add a new menu that branches off of the existing menu entry. Double-click the `Type Here` placeholder and enter the text for the new item. Refer to Figure 7-19.

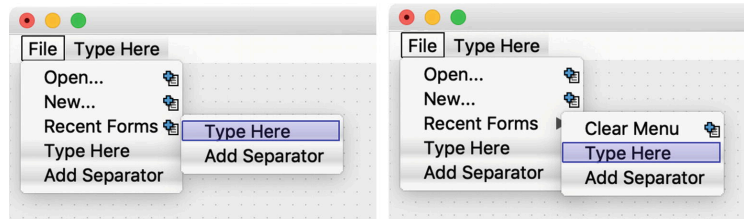


Figure 7-19 Adding submenus. Click the plus symbol next to menu entry (left). Add new entry (right)

Adding Toolbars in Qt Designer

Toolbars can be added to the main window by right-clicking the form to open a context menu. Click the `Add Tool Bar` option.

The actions in toolbars are created as toolbar buttons and can be dragged between the menus and the toolbar. You can also add icons to the toolbar. This topic is covered in the “Display Images in Qt Designer” section. An example of the toolbar with an icon is shown in Figure 7-20.

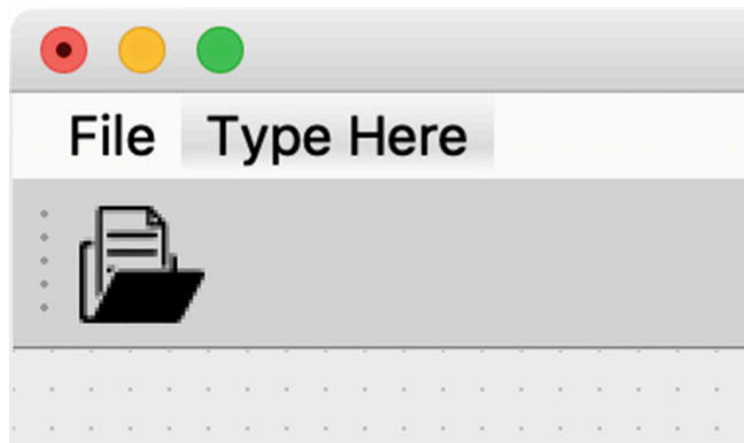


Figure 7-20 Toolbar with Open toolbar button

Adding Actions in Qt Designer

When items are first created in the menu and the toolbar, they are actually actions. Actions can be created, removed, given an icon, designated a shortcut hot key, and made checkable all in the `Action Editor` dock widget (shown in Figure 7-7). Actions can also be shared between the menu and the toolbar.

To share an action between the menu and the toolbar so that both objects contain the same item, drag the action from the `Action Editor` that already exists in the menu onto the toolbar.

Display Images in Qt Designer

This last section will take a quick look at how you can include images and icons in your application. Whether you are looking to add an image to a QLabel widget or trying to add icons to your toolbar, the process for adding an image is similar.

For example, if you have a QLabel widget on your form, you can access its properties in the Property Editor, shown below in Figure 7-21. Scroll down until you find the pixmap property. Click its Value and from here you will be able to search for an image file. If you want to add an icon, then you will look for the icon property, not pixmap.

You are given two options: Choose Resource... and Choose File... If you have added resources to your project, then select Choose Resource... Otherwise, you can search for images on your computer.

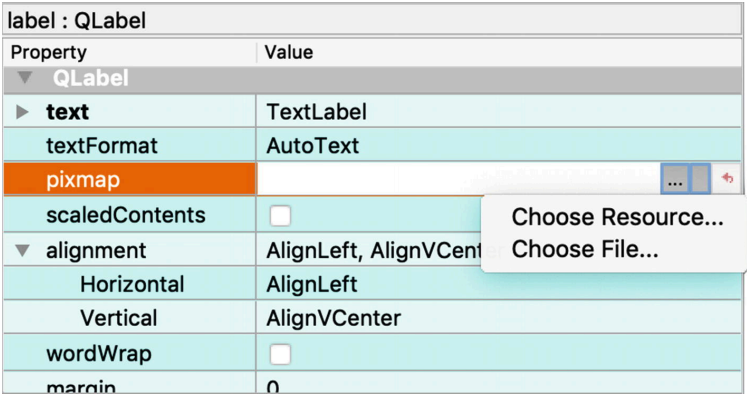


Figure 7-21 Add images to your application using the pixmap property

Summary

Qt Designer is definitely a useful tool for creating GUI applications. It provides a drag and drop interface that makes it easier to lay out widgets; modify the parameters of objects; create menus, toolbars, and dock widgets; add actions to menus; generate code that can be used in Python; and more. Qt Designer can make the design process much quicker and easier.

While this chapter covered a few of the basics for using Qt Designer, there are still other uses such as creating your own custom widgets that can be included in Qt Designer or generating dialog boxes.

The following chapters will begin to look at more specific classes that can be used to further augment a user interface. Chapter 8 takes a look at the QClipboard class and creating widgets with drag and drop functionality.